

LAPORAN TEKNIS · MATERI PEMBELAJARAN

# Agentic AI Dari Model Bahasa ke Agen Otonom

Sebuah eksplorasi menyeluruh tentang cara kerja agentic AI — dari fondasi teoretis, spesifikasi arsitektur, implementasi praktis, hingga studi kasus dunia nyata.

Materi Pembelajaran

Versi 1.0 · 2025.06

Proyek Agentic Research

# | Daftar Isi

|    |                                    |    |
|----|------------------------------------|----|
| 01 | Ringkasan Eksekutif                | 03 |
| 02 | Teorema: Fondasi Agentic AI        | 04 |
| 03 | Spesifikasi: Arsitektur & Komponen | 07 |
| 04 | Praktikal: Implementasi & Workflow | 10 |
| 05 | Use Case: Aplikasi Dunia Nyata     | 12 |
| 06 | Kesimpulan & Referensi             | 14 |

## 01 · RINGKASAN EKSEKUTIF

# | Ringkasan Eksekutif

Sebuah **LLM tradisional** adalah mesin tanya-jawab: satu prompt masuk, satu jawaban keluar. Sebuah **agentic AI** bekerja secara fundamental berbeda — ia bernalar tentang apa yang dibutuhkan, memanggil tools untuk mengumpulkan informasi, mengingat konteks lintas giliran, dan melakukan iterasi hingga bisa memberikan jawaban yang lengkap. Dokumen ini menjelaskan **fondasi teoretis** yang memungkinkan transisi ini, dari tingkat teori hingga praktik.

## Poin-Poin Kunci

- LLM tidak bersifat agentic secara inheren. Agency muncul ketika kita menggabungkan **instruction following**, **definisi tools**, dan sebuah **decision loop**.
- Pola ReAct (Reason + Act) adalah algoritma fundamental — model berpikir, memutuskan untuk menggunakan tool atau merespons, bertindak, mengamati hasil, dan mengulangi.
- Empat komponen membentuk sistem agentic yang lengkap: **LLM** (penalaran), **Tools** (tindakan), **Memory** (konteks), dan **RAG** (pengambilan pengetahuan).
- Dengan building block ini, agen dapat melakukan riset, menulis kode, menangani dukungan pelanggan, dan menganalisis data — secara otonom dan iteratif.

### PERTANYAAN YANG DIJAWAB DOKUMEN INI

Bagian apa dari LLM yang memungkinkan perilaku agentic? Bagaimana loop ReAct menghubungkan penalaran dengan tindakan? Dan seperti apa sistem agentic yang lengkap dalam praktiknya — dari arsitektur hingga workflow nyata?

# | Fondasi Agentic AI

Sebelum memahami agentic AI, kita perlu memahami apa yang sebenarnya terjadi di dalam LLM. Sebuah language model, pada intinya, adalah mesin prediksi token-berikutnya (next-token prediction). Diberikan urutan teks, ia memprediksi token paling mungkin berikutnya. Menskalakan mekanisme sederhana ini ke miliaran parameter menghasilkan perilaku yang sangat canggih — tetapi agency membutuhkan lapisan kemampuan spesifik di atasnya.

## Empat Lapisan yang Memungkinkan Agency

### 1. Pre-training — Fondasi Pengetahuan

Selama pre-training, model menyerap data teks dalam jumlah masif dan mempelajari pola statistik: fakta, rantai penalaran, sintaks kode, gaya penulisan. Ini adalah pengetahuan dasar model — segala sesuatu yang ia "ketahui" sebelum tuning apapun. Namun, sebuah base model tidak bisa mengikuti instruksi atau menggunakan tools secara andal. Ia hanya memprediksi apa yang muncul berikutnya.

### 2. Instruction Tuning — Mengajarkan Model Mengikuti Arahan

Instruction tuning melatih model pada pasangan (prompt, respons) di mana respons mengikuti format instruksi tertentu. Ini adalah langkah kritis yang mengubah mesin pelengkap teks menjadi asisten yang mengikuti perintah. Tanpa langkah ini, "cari di web tentang agen AI" mungkin dilengkapi sebagai "dan temukan artikel relevan" — bukan dipahami sebagai perintah untuk melakukan tindakan.

### 3. Function Calling — Jembatan ke Tindakan Eksternal

Ini adalah kemampuan paling kritis untuk agency. Function calling mengajarkan model untuk menghasilkan JSON terstruktur yang mendeskripsikan tool mana yang akan dipanggil dan dengan parameter apa, alih-alih teks bebas. Sistem menangkap JSON ini, mengeksekusi fungsi yang sesungguhnya, dan mengembalikan hasilnya. Inilah cara "pikiran" LLM menjadi "tindakan" di dunia nyata.

```
// Saat LLM memutuskan untuk mencari, ia menghasilkan:
{
  "tool_call": "web_search",
  "arguments": { "query": "apa itu agen AI" }
}

// Sistem mengeksekusi pencarian dan mengembalikan:
{
  "role": "tool",
  "content": "Hasil pencarian: 1. Agen AI adalah..."
}
```

Function calling bukan fitur opsional — ia adalah fondasi. Tanpa kemampuan ini, model hanya bisa "berbicara" — tidak pernah "bertindak." Function calling adalah interface yang memungkinkan LLM dan environment eksekusi berkomunikasi secara terstruktur.

4. Context Window — Memori Kerja Model

LLM modern memiliki context window berkisar dari 8K hingga 200K+ token — pada dasarnya **memori jangka pendek** yang menampung seluruh percakapan. Setiap pesan, panggilan tool, dan hasil tool ditambahkan ke window ini. Model melihat semua yang telah terjadi sejauh ini, memungkinkannya membangun dari langkah-langkah sebelumnya tanpa kehilangan jejak.

Apa yang Sebenarnya "Dilihat" oleh Model

Selama agent loop berjalan, input penuh ke LLM adalah daftar pesan yang dibangun secara hati-hati:

| Role      | Konten                                                                                                                  | Tujuan                                                         |
|-----------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| System    | Anda adalah agen riset. Anda memiliki tools: web_search, read_webpage, save_note, recall_note, think. Ikuti pola ReAct. | Mendefinisikan identitas agen, aturan, dan tools yang tersedia |
| User      | Riset tentang agen AI dan bandingkan dengan RPA.                                                                        | Tugas yang harus diselesaikan                                  |
| Assistant | Saya akan mencari di web dulu. Memanggil web_search("agen AI vs RPA")                                                   | Penalaran + tindakan model                                     |
| Tool      | Hasil pencarian: 1. Apa itu agen AI... 2. RPA vs AI...                                                                  | Output tool, dimasukkan kembali ke konteks                     |
| Assistant | Berdasarkan hasil pencarian, agen AI bernalar sementara RPA mengikuti skrip. Saya akan menyimpan temuan ini.            | Model memproses hasil dan memutuskan langkah berikutnya        |

Daftar pesan ini adalah **seluruh realitas** tempat model beroperasi. Kualitas system prompt, kejelasan definisi tools, dan struktur riwayat percakapan secara langsung menentukan kualitas perilaku agen.

Jika prompt sistem buruk, agen akan bingung. Jika definisi tools tidak jelas, agen akan salah memanggil. Jika konteks terlalu panjang, agen akan "lupa."

Agency bukanlah sihir. Ia adalah kombinasi cermat dari instruction following (agar model patuh), function calling (agar model bisa bertindak), dan manajemen konteks (agar model ingat). Setiap lapisan dibangun di atas lapisan sebelumnya — dan semuanya diperlukan.

## Mengapa LLM Bisa Bernalar?

Pada level fundamental, LLM tidak "bernalar" seperti manusia. Ia melakukan **next-token prediction** — diberikan urutan token, prediksi token berikutnya. Namun, pada skala miliaran parameter yang dilatih di atas data berkualitas tinggi (buku, makalah ilmiah, kode, percakapan), prediksi token-berikutnya menghasilkan output yang **terlihat seperti penalaran**. Model belajar bahwa setelah "langkah pertama, kita perlu..." biasanya diikuti oleh rencana yang koheren. Setelah "definisi X adalah..." diikuti oleh definisi yang akurat. Ini adalah **statistical reasoning**, bukan symbolic reasoning — tetapi hasilnya cukup powerful untuk membangun agentic behavior.

## 03 · SPESIFIKASI

# Arsitektur & Komponen Agentic AI

Setelah memahami fondasi teoretis, kita masuk ke [spesifikasi arsitektur](#). Sebuah sistem agentic yang lengkap terdiri dari lima komponen yang diorkestrasi oleh Agent Loop pusat. Setiap komponen memiliki tanggung jawab yang jelas dan terpisah.

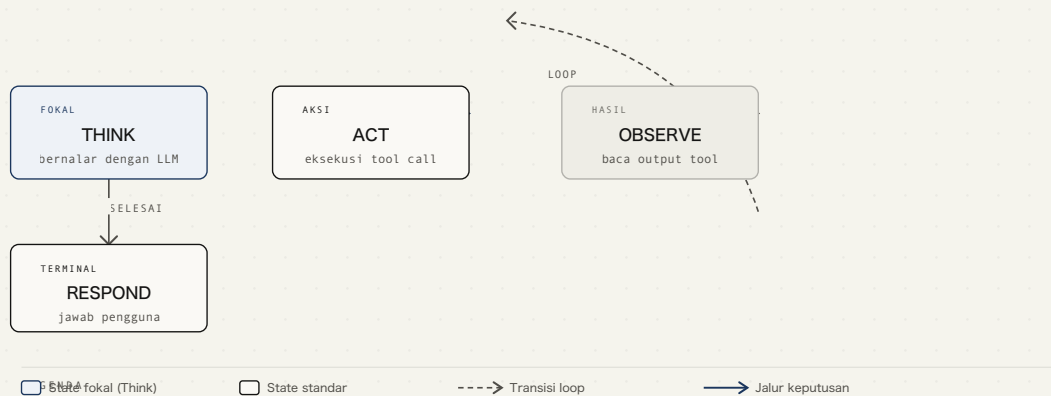
## Pola ReAct — Algoritma Inti

ReAct (Reasoning + Acting) adalah [algoritma fundamental](#) di balik hampir semua sistem agentic AI. Ia menyisipkan langkah berpikir (thinking) dengan langkah bertindak (acting), memungkinkan model mengumpulkan informasi, merefleksikannya, dan merencanakan langkah berikutnya.

Agent berputar melalui empat state hingga bisa memberikan jawaban lengkap:

1. **THINK** — LLM menerima konteks penuh (system prompt, riwayat percakapan, hasil tool) dan bernalar tentang apa yang harus dilakukan selanjutnya.
2. **ACT** — Jika model menghasilkan tool call, sistem menangkapnya dan mengeksekusi fungsi Python yang sesuai.
3. **OBSERVE** — Output tool dibaca dan ditambahkan ke percakapan sebagai pesan "tool" baru. Model sekarang memiliki informasi baru.
4. **RESPOND** — Ketika model menentukan sudah memiliki cukup informasi, ia menghasilkan jawaban teks final tanpa memanggil tools lagi.

GAMBAR 1 LOOP REACT AGENT



Gambar 1. Loop ReAct memiliki empat state. Agen berputar melalui Think → Act → Observe hingga memiliki informasi cukup untuk Respond. Garis putus-putus dari Observe kembali ke Think adalah yang membuat sistem bersifat iteratif.

## Pseudocode Implementasi

```
def react_loop(query, tools, max_steps=8):
    messages = [system_prompt_dengan_tools(tools)]
    messages.append({"role": "user", "content": query})

    for step in range(max_steps):
        response = llm.chat(messages)      # THINK

        if response.memiliki_tool_call():  # DECIDE
            result = eksekusi_tool(        # ACT
                response.nama_tool,
                response.argumen_tool
            )
            messages.append(hasil_tool)    # OBSERVE
            continue                      # LOOP

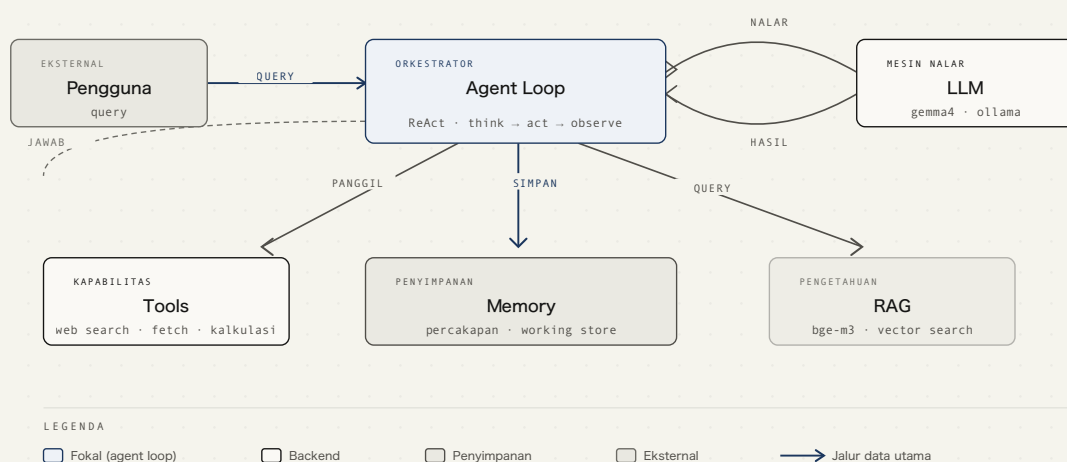
    return response.konten                # RESPOND

return "Batas langkah tercapai"
```

Insight kunci: LLM dipanggil berulang kali — bukan sekali. Setiap panggilan melihat riwayat percakapan yang terus bertambah. Ini secara fundamental berbeda dari chatbot single-turn. Kualitas "agentik" muncul dari loop, bukan dari kemampuan model tunggal manapun.

## Arsitektur Sistem — Lima Komponen

GAMBAR 2 ARSITEKTUR SISTEM AGENTIC AI



Gambar 2. Empat komponen mengorbit Agent Loop: LLM menyediakan penalaran, Tools memungkinkan aksi eksternal, Memory menjaga konteks, dan RAG mengambil pengetahuan dari vector store yang terindeks.

## Matriks Spesifikasi Komponen

| Komponen   | Peran                                                                                              | Implementasi                              | Properti Kunci                               |
|------------|----------------------------------------------------------------------------------------------------|-------------------------------------------|----------------------------------------------|
| Agent Loop | Orkestrator — mengelola siklus ReAct, merutekan pesan, memberlakukan batas langkah                 | Loop Python dengan max_step guard         | Mesin iterasi stateless                      |
| LLM        | Mesin penalaran — memutuskan tool apa yang dipanggil atau kapan merespons                          | Ollama API (gemma4, llama3, dll.)         | Token-in, token-out dengan dukungan tool     |
| Tools      | Lapisan aksi — fungsi yang bisa dipanggil LLM: search, fetch, save, kalkulasi                      | Fungsi Python dengan skema JSON           | Setiap tool punya nama, deskripsi, parameter |
| Memory     | Persistensi konteks — riwayat percakapan (jangka pendek) dan working store (sesi penuh)            | Daftar pesan + dict penyimpanan key-value | Dua tingkat: history + working memory        |
| RAG        | Pengambilan pengetahuan — embed dokumen, cari berdasarkan similaritas, kembalikan potongan relevan | Model embedding (bge-m3) + vector DB      | Cosine similarity pada vektor 1024-dimensi   |

Separation of concerns. LLM tidak tahu cara mencari di web. Tools tidak tahu kapan harus dipanggil. Memory tidak tahu apa yang harus diingat. Hanya Agent Loop — melalui pola ReAct — yang menghubungkan semuanya menjadi sistem yang koheren.

## 04 · PRAKTIKAL

# Implementasi & Workflow Agentic AI

Setelah memahami teori dan spesifikasi, kita masuk ke **implementasi praktis**. Berikut adalah workflow agentic AI yang sesungguhnya — setiap langkah dengan anotasi state, aksi, dan hasil yang bisa diamati. Ini adalah **apa yang sebenarnya terjadi** saat Anda menjalankan agen.

## Workflow 1 — Agen Riset Menjawab Pertanyaan

Query pengguna: "Apa itu agen AI dan bagaimana perbedaannya dengan RPA?"

| Langkah | State   | Aksi                                                                                                           | Hasil yang Teramati                                       |
|---------|---------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| 1       | THINK   | LLM bernalar: "Saya tidak punya pengetahuan terkini. Saya perlu mencari."                                      | Penalaran internal — tidak ada output eksternal           |
| 2       | ACT     | Memanggil <code>web_search("agen AI vs RPA")</code>                                                            | Mengembalikan 5 hasil pencarian dengan judul dan cuplikan |
| 3       | OBSERVE | Hasil ditambahkan ke percakapan                                                                                | LLM sekarang memiliki data baru dalam konteks             |
| 4       | THINK   | LLM bernalar: "Hasil 1 terlihat menjanjikan. Saya akan baca artikel penuhnya."                                 | Merencanakan tool call berikutnya                         |
| 5       | ACT     | Memanggil <code>read_webpage("https://example.com/agen-ai")</code>                                             | Mengembalikan teks artikel penuh (dipotong 4000 karakter) |
| 6       | OBSERVE | Konten artikel ditambahkan ke percakapan                                                                       | LLM membaca penjelasan lengkap                            |
| 7       | ACT     | Memanggil <code>save_note("agen_vs_rpa", "Agen AI bernalar dan beradaptasi; RPA mengikuti skrip tetap")</code> | Memory: kunci disimpan untuk dipanggil nanti              |
| 8       | THINK   | LLM bernalar: "Saya sudah cukup. Saya akan sintesis."                                                          | Memutuskan untuk merespons                                |
| 9       | RESPOND | Menghasilkan jawaban final yang menyintesis hasil pencarian, konten artikel, dan catatan tersimpan             | Jawaban komprehensif dikirim ke pengguna                  |

## Workflow 2 — Multi-turn Memory Recall

Query pengguna: "Tadi kamu menyebut RPA. Bisakah jelaskan lebih detail tentang keterbatasannya?"

| Langkah | State           | Aksi                                                                                                      | Hasil yang Teramati                                                      |
|---------|-----------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| 1       | THINK           | LLM membaca riwayat percakapan: melihat riset sebelumnya tentang agen AI, catatan tersimpan "agen_vs_rpa" | Konteks dari giliran sebelumnya tersedia                                 |
| 2       | ACT             | Memanggil <code>recall_note("agen_vs_rpa")</code>                                                         | Mengambil: "Agen AI bernalar dan beradaptasi; RPA mengikuti skrip tetap" |
| 3       | THINK           | LLM bernalar: "Saya perlu mencari informasi lebih spesifik tentang keterbatasan RPA."                     | Merencanakan pencarian yang ditargetkan                                  |
| 4       | ACT             | Memanggil <code>web_search("keterbatasan RPA skalabilitas 2025")</code>                                   | Mengembalikan artikel tentang kelemahan RPA                              |
| 5       | THINK → RESPOND | Mensintesis memori yang dipanggil + hasil pencarian baru menjadi jawaban detail                           | Jawaban didasarkan pada konteks sebelumnya dan riset baru                |

Mengapa ini penting. Workflow 2 menunjukkan sinergi **memory + tools**. Agen tidak memulai dari nol — ia mengingat temuan sebelumnya, lalu menggunakan tools untuk mengisi kekosongan. Ini yang membedakan agen dari chatbot single-turn.

## Workflow 3 — RAG-Powered Knowledge Building

Skenario: Agen menyimpan temuan riset ke knowledge base selama sesi, lalu mencarinya untuk referensi silang.

```
# Langkah A: Agen menyimpan temuan ke knowledge base
save_note("definisi_rpa", "RPA mengotomatisasi tugas repetitif via skrip UI")
rag.add("RPA mengotomatisasi tugas repetitif via skrip UI") # di-embed dengan
bge-m3

save_note("definisi_agen", "Agen AI bernalar, merencanakan, dan menggunakan tools
secara otonom")
rag.add("Agen AI bernalar, merencanakan, dan menggunakan tools secara otonom")

# Langkah B: Nanti, agen mencari di knowledge base
results = rag.search("otomatisasi yang bisa bernalar") # cosine similarity
# → Mengembalikan "Agen AI bernalar, merencanakan..." (skor: 0.87)
# → Mengembalikan "RPA mengotomatisasi tugas..." (skor: 0.42)

# Langkah C: Agen menggunakan hasil RAG untuk menjawab pertanyaan baru
# "Otomatisasi seperti apa yang bisa beradaptasi dengan input tak terduga?"
# LLM melihat hasil RAG dalam konteks → menjawab dengan percaya diri
```

Sistem RAG mengubah agen dari **peneliti sekali pakai** menjadi **pembelajar kumulatif**. Setiap temuan memperkaya knowledge base, dan query berikutnya mendapat manfaat dari semua yang telah ditemukan — tanpa mencari ulang di web.

## 05 · USE CASE

# | Aplikasi Dunia Nyata

Pola agentik bukan sekadar keingintahuan akademis — ia mendukung **sistem produksi** di berbagai bidang: riset, pengembangan software, layanan pelanggan, dan analisis data. Berikut adalah lima kategori dengan contoh konkret.

## 1. Riset & Sintesis Pengetahuan

TOOLS

MEMORY

RAG

Agen mencari dari berbagai sumber, membaca artikel penuh, mengekstrak fakta kunci, menyimpannya ke memori, dan mensintesis laporan komprehensif. Tidak seperti mesin pencari yang hanya mengembalikan tautan, agen **menghasilkan sintesisnya sendiri**. Use case mencakup tinjauan literatur, analisis kompetitif, riset pasar, dan due diligence teknis.

```
# Workflow agen riset
query = "Bandingkan GPT-4, Claude, dan Gemini pada benchmark penalaran"
# → web_search → read_webpage × 3 → save_note × 5 → think → respond
# Output final: tabel perbandingan terstruktur dengan kutipan
```

## 2. Code Generation & Debugging

TOOLS

MEMORY

Coding agent (GitHub Copilot, Cursor, Claude Code) menggunakan pola yang sama: baca codebase, rencanakan perubahan, hasilkan kode, jalankan tes, amati hasil, dan iterasi. Loop **eksekusi → amati → perbaiki** adalah aplikasi langsung dari ReAct.

```
# Workflow coding agent
query = "Tambahkan rate limiting ke endpoint API"
# → read_file("routes.py") → think("Perlu middleware") → tulis_kode
# → jalankan_tes → amati: 2 tes gagal → perbaiki → tes ulang → respond: "Selesai, 4/4 lulus"
```

## 3. Layanan Pelanggan

TOOLS

MEMORY

RAG

Agen dukungan pelanggan menarik dari knowledge base (RAG), memeriksa status pesanan (tools/API), mengingat interaksi sebelumnya (memory), dan melakukan eskalasi jika diperlukan. Agen **menangani alur resolusi penuh** — bukan hanya menyarankan artikel bantuan.

## 4. Analisis Data & Pelaporan

### TOOLS

### MEMORY

Agen menerima dataset, merumuskan hipotesis, menulis dan mengeksekusi query, memvisualisasikan hasil, dan menulis ringkasan — semua dalam satu sesi. Setiap langkah [memberi masukan ke langkah berikutnya](#) melalui riwayat percakapan.

```
# Workflow analisis data
query = "Analisis penjualan Q3 dan identifikasi region dengan performa terbaik"
# → jalankan_query("SELECT region, SUM(revenue) FROM sales WHERE quarter=3...")
# → amati: Barat +32%, Timur -5%
# → think("Pertumbuhan Barat didorong oleh apa?")
# → jalankan_query("SELECT product, revenue FROM sales WHERE region='Barat'...")
# → respond: analisis terstruktur dengan data
```

## 5. Asisten Pribadi & Otomatisasi Tugas

### TOOLS

### MEMORY

Agen dapat mengelola kalender, menyusun email, merangkum rapat, dan mengeksekusi workflow multi-langkah. "Jadwalkan rapat dengan tim Selasa depan, kirim agenda, dan ingatkan saya satu jam sebelumnya" — agen [merantai beberapa tool call](#) untuk menyelesaikan tugas kompleks secara otonom.

## Matriks Kapabilitas per Use Case

| Use Case          | Penalaran LLM | Tools                | Memory | RAG      | Kompleksitas |
|-------------------|---------------|----------------------|--------|----------|--------------|
| Riset             | Berat         | Search, fetch        | Ya     | Ya       | Menengah     |
| Code generation   | Berat         | File I/O, exec, test | Ya     | Opsional | Tinggi       |
| Layanan pelanggan | Menengah      | API calls, lookup    | Ya     | Ya       | Menengah     |
| Analisis data     | Berat         | SQL, chart, stats    | Ya     | Opsional | Tinggi       |
| Otomatisasi tugas | Ringan        | Kalender, email, API | Ya     | Tidak    | Rendah       |

Polanya universal. Di semua lima use case, loop ReAct yang sama berjalan. Yang berubah adalah set tools — yang persis merupakan tujuan desainnya. Arsitektur agen bersifat tool-agnostic; Anda menukar tools untuk mengubah domain.

# | Kesimpulan & Referensi

Agentic AI bukanlah satu teknologi tunggal — ia adalah **pola arsitektur** yang menggabungkan mesin penalaran, tools eksternal, memori persisten, dan pengambilan pengetahuan ke dalam loop pengambilan keputusan otonom. Dari teori hingga praktik, pola ini konsisten.

## Kesimpulan Inti

1. LLM diperlukan tapi tidak cukup. LLM mentah tidak bisa mencari di web, mengingat giliran sebelumnya di luar context window, atau mengambil dokumen. Agency membutuhkan **lapisan orkestrasi** — Agent Loop — yang menghubungkan model ke tools, memory, dan knowledge base.
2. ReAct adalah pola universal. Think → Act → Observe adalah cara setiap sistem agentik bekerja di balik layar. Memahami loop ini adalah memahami agentic AI.
3. Empat komponen, tak terbatas aplikasi. LLM + Tools + Memory + RAG membentuk **arsitektur yang dapat dikomposisi**. Tukar tools dan Anda mendapatkan agen yang berbeda: riset, coding, dukungan, analisis — loop sama, kapabilitas berbeda.
4. Function calling adalah enabler. Kemampuan LLM untuk menghasilkan tool call terstruktur (bukan teks bebas) adalah kapabilitas paling penting yang membuat agen praktis. Tanpanya, model dan lingkungan eksekusi tidak bisa berkomunikasi.
5. Memory dan RAG menciptakan nilai kumulatif. Agen stateless memulai dari nol setiap giliran. Agen dengan working memory dan vector knowledge base **mengakumulasi kecerdasan** sepanjang sesi — setiap temuan memperkaya respons berikutnya.

## Fondasi Teoretis

Pola agentik berasal dari beberapa jalur riset yang mapan:

- ReAct (Yao et al., 2023) — Menyisipkan penalaran dan pembangkitan aksi dalam language model.
- Toolformer (Schick et al., 2023) — Mengajarkan LLM menggunakan tools eksternal via self-supervised learning.
- RAG (Lewis et al., 2020) — Retrieval-augmented generation untuk tugas NLP yang intensif pengetahuan.
- Chain-of-Thought (Wei et al., 2022) — Memperoleh penalaran langkah-demi-langkah melalui prompting.
- Function Calling (OpenAI, 2023) — API tool-use terstruktur yang memungkinkan LLM memanggil fungsi eksternal.

### GAMBARAN BESAR

LLM berpikir. Tools menjangkau melampaui data pelatihan. Memory mengingat lintas giliran. RAG mengambil pengetahuan sesuai permintaan. Agent Loop menghubungkan semuanya — dan hasilnya adalah AI yang tidak hanya menjawab pertanyaan, tetapi mengumpulkan informasi, membuat keputusan, dan mengambil tindakan.

## Referensi

- Yao, S. et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR 2023*.
- Schick, T. et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." *NeurIPS 2023*.
- Lewis, P. et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS 2020*.
- Wei, J. et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *NeurIPS 2022*.
- Ollama — Runtime LLM lokal. [ollama.com](https://ollama.com).
- bge-m3 (BAAI) — Model embedding multibahasa. [huggingface.co/BAAI/bge-m3](https://huggingface.co/BAAI/bge-m3).
- Gemma (Google DeepMind) — Keluarga model bahasa open-weight. [ai.google.dev/gemma](https://ai.google.dev/gemma).

## Ucapan Terima Kasih

Dokumen ini dibuat sebagai materi pembelajaran untuk memahami cara kerja sistem agentic AI, dari fondasi teoretis hingga implementasi praktis. Kode pendamping di [sinau-agentic/](https://github.com/sinau-agentic/) menyediakan implementasi yang berfungsi dari agentic research AI menggunakan model lokal (Ollama + gemma4 + bge-m3), mendemonstrasikan setiap konsep yang dibahas dalam dokumen ini.